ShortList

Build it in Claude Code — step by step, from scratch

A plain-language playbook that takes you from an empty folder to a deployed, BYOK AI shortlisting engine. Every step tells you what you are doing, the exact prompt to give Claude Code, and how to check it worked before moving on.

Companion to: CLAUDE.md (the spec) and BUILD_PLAN.md (the roadmap)
Owner: Ankit · Updated: 4 June 2026

How to use this guide

IN PLAIN TERMS

Claude Code is an AI that writes and runs the code for you. Your job is not to type code — it is to tell Claude Code clearly what to build, then check its work. This guide gives you both: the exact words to say, and the exact thing to look at afterwards.

The loop you will repeat for every step:

- **Set context once per session** — tell Claude Code to read your CLAUDE.md and BUILD_PLAN.md first.
- **Give it one step at a time** — paste the prompt from this guide. Never paste a whole phase at once.
- **Let it build and run** — it will write files, install packages, and run them.
- **Check the green box** — do the 'Check before moving on' test yourself. If it fails, tell Claude Code exactly what you saw and let it fix it.
- **Commit** — say “commit this with a clear message” so you can always go back.

CHECK BEFORE MOVING ON

Golden rule: if you don't understand what a step did, ask Claude Code “explain what you just built in simple terms and how I can test it myself.” Never move on while confused.

Part 0 — Before you write any code

Accounts, tools, and the project skeleton. ~half a day.

Step 1. Install the tools on your computer

IN PLAIN TERMS

You need three things on your machine: Node.js (so Claude Code can run), Git (to save versions), and Claude Code itself.

RUN IN TERMINAL

```
# install Node.js LTS from nodejs.org first, then:
npm install -g @anthropic-ai/claude-code
claude --version
```

CHECK BEFORE MOVING ON

Running `claude --version` prints a version number with no error.

Step 2. Open the accounts you'll need

- **GitHub** — to store your code.
- **Supabase** — your database, login system, and file storage, all in one.
- **Render** (or Railway / Fly) — to put your app on the internet.
- **OpenRouter** — one connection that reaches Claude, OpenAI, Gemini, etc. Get a test key (yours, for building only).
- **Claude Max subscription** — powers your Claude Code usage during the build.

CHECK BEFORE MOVING ON

You can log in to each, and you have an OpenRouter test key copied somewhere safe.

Step 3. Create the project folder and start Claude Code

RUN IN TERMINAL

```
mkdir shortlist && cd shortlist
git init
claude
```

IN PLAIN TERMS

From now on you talk to Claude Code in plain English inside this folder. First, hand it the brief and the plan so it knows the whole picture.

PASTE THIS TO CLAUDE CODE (then paste the two documents)

```
Read CLAUDE.md and BUILD_PLAN.md in this folder (I will paste them now). This is the product we are building together. From now on, follow that spec. Keep all secrets, API keys, and model names in environment config — never hard-coded. After each task, write a short note in a PROGRESS.md file describing what you did and how I can test it.
```

CHECK BEFORE MOVING ON

Claude Code confirms it has read both files and can summarise the product back to you correctly.

Phase 0 — Foundations & the two-service skeleton

Two empty apps that can log in and safely store a key. ~1 week.

IN PLAIN TERMS

You're building the frame of the house: the two services from the spec (the public 'intake' app where candidates apply, and the private 'processing' app where the AI works), a database, a login, and a locked safe for customer API keys. No AI yet.

Step 4. Create the two services and the database

PASTE THIS TO CLAUDE CODE

```
Set up the project from Phase 0 of the plan. Create two FastAPI services:
'intake-service' and 'processing-service', in one repo. Connect to my Supabase project
(I'll give you the keys as env vars). Create the database tables exactly as listed in
section 12 of CLAUDE.md, with row-level security so each org can only read its own rows.
Add a README explaining how to run both locally.
```

CHECK BEFORE MOVING ON

You can start both services locally and open their health-check pages in a browser.

Step 5. Add recruiter login

PASTE THIS TO CLAUDE CODE

```
Add recruiter sign-up and login to the processing-service using Supabase auth. A
logged-in recruiter belongs to one org. Show me a simple page that says who is logged in
and which org they belong to.
```

CHECK BEFORE MOVING ON

You can register, log in, and the page shows your org. A second test account cannot see the first's data.

Step 6. Build the locked safe for API keys (the key vault)

IN PLAIN TERMS

Customers will paste their own API key. It must be encrypted so that even you can't read it, never written to logs, and only ever used to make their searches.

PASTE THIS TO CLAUDE CODE

```
Build the BYOK key vault from the spec. Let an org save an API key; encrypt it at rest
using envelope encryption; never log it or return it in plaintext. Add a 'test key'
button that makes one small OpenRouter call to confirm the key works, and shows only
success/failure — never the key.
```

CHECK BEFORE MOVING ON

After saving a key, search the logs and database — you should never find the raw key, only encrypted text. The 'test key' button works.

Step 7. Connect the two services and put them online

PASTE THIS TO CLAUDE CODE

```
Add a routing channel so the intake-service can hand a new application to the
processing-service (use a queue or webhook). Send one dummy application across to prove
it works. Then set up automatic deployment: when I push to GitHub, both services deploy
to Render staging.
```

CHECK BEFORE MOVING ON

A dummy application sent from intake shows up in processing, and pushing to GitHub auto-deploys both.

Phase 0 done when: both services are online, a recruiter can log in, a key is stored encrypted and validated, and a message routes between the services.

Phase 1 — Intake service (candidates apply)

A candidate opens a link, drops a CV, it lands grouped by job. ~1 week.

IN PLAIN TERMS

This is the only part the public sees. Keep it dead simple and make sure it never touches any API keys.

Step 8. Let recruiters create a job with a shareable link

PASTE THIS TO CLAUDE CODE

```
In the processing-service, let a recruiter create a Job (this is the 'Application name'). Generate a unique public apply-link for each job that I can share. List a job's applications in the recruiter view.
```

CHECK BEFORE MOVING ON

You can create a job and copy its apply-link.

Step 9. Build the candidate apply screens

PASTE THIS TO CLAUDE CODE

```
In the intake-service, build the candidate application screens in React for a given apply-link: a few minimal fields plus a CV upload (PDF or DOCX). On submit, store the file in Supabase storage, create an application grouped under that job, and route it to the processing-service. Add file-type and size limits and basic rate limiting. This service must hold no API keys.
```

CHECK BEFORE MOVING ON

Open an apply-link on your phone, submit a CV, and confirm it appears under the right job in the recruiter view.

CHECK BEFORE MOVING ON

Submit 20 test applications — all land, all grouped correctly, none lost.

Phase 2 — Parsing pipeline (CV to clean text)

Turn messy CV files into clean text the AI can read. ~1 week.

IN PLAIN TERMS

CVs come in every shape — two columns, tables, even scanned images. This step cleans them up so the AI isn't reading garbage. Bad parsing here quietly ruins every score later, so test it hard.

Step 10. Extract and clean every CV

PASTE THIS TO CLAUDE CODE

```
Build the parsing pipeline from Phase 2. For each uploaded CV, extract clean text from PDF and DOCX, handling multi-column layouts and tables, with an OCR fallback for scanned/image PDFs. Save the clean text on the candidate record. If a CV barely extracts any text, flag it as 'needs review' instead of passing empty text forward.
```

CHECK BEFORE MOVING ON

Feed it 15-20 deliberately ugly real CVs (multi-column, scanned, tables). Read the cleaned output — it should be readable, and the broken ones should be flagged, not silently empty.

Phase 3 — The scoring engine (the core AI)

Score each CV on its own, with cited evidence. ~1.5-2 weeks. The heart of the product.

IN PLAIN TERMS

This is where the magic happens — and where most tools go wrong. The trick from your spec: score each CV completely on its own (like grading exam papers one at a time), so the AI never mixes up one candidate with another. Every score must point to a real quote from that CV, and the AI must be allowed to say 'this CV doesn't show that' instead of guessing.

Step 11. Turn the recruiter's request into a scoring rubric

PASTE THIS TO CLAUDE CODE

```
Build the rubric step from Phase 3. Take the recruiter's plain-English prompt plus the attached JD (and optional weights) and turn them into a fixed list of scoring criteria with weights. Show the recruiter the rubric before scoring so they can tweak it.
```

CHECK BEFORE MOVING ON

Give it a real prompt + JD; the criteria it produces actually reflect what the role needs.

Step 12. Score each CV independently, with evidence

PASTE THIS TO CLAUDE CODE

```
Build the independent scoring step. For each candidate, make ONE model call (via OpenRouter, model chosen from org config) that scores only that CV against the rubric and returns exactly the JSON schema in section 7.4 of CLAUDE.md: per-criterion score, a real evidence_quote copied from the CV, an insufficient_evidence flag, a short summary, and flags. Validate the JSON every time; retry on bad output. Never put two candidates in the same call.
```

THEN PASTE THIS (cost optimisation)

```
Now optimise cost exactly as the spec says: cache the shared JD/rubric across all CVs in a search, run the scoring calls in parallel, and add a config switch for a cheap model (or embeddings) on the first pass and a stronger model later. Keep all model names in config.
```

CHECK BEFORE MOVING ON

Score 50 CVs. Open ten results and confirm each evidence_quote is actually present word-for-word in that candidate's CV, and that 'insufficient_evidence' fires when a CV genuinely lacks something.

CHECK BEFORE MOVING ON

Confirm no candidate's evidence ever comes from someone else's CV. This is your hallucination test.

Phase 4 — Rank, refine & the recruiter dashboard

Turn scores into a shortlist and prove 300 to top 10. ~1.5 weeks.

IN PLAIN TERMS

Now you stack-rank. The ordering is done by plain maths on the scores (no AI, so no mistakes), then the AI takes a careful second look only at the top handful. Then you prove the whole thing works on a real 300-CV pile.

Step 13. Rank by score, then refine the top of the list

PASTE THIS TO CLAUDE CODE

```
Build ranking from Phase 4. Sort all candidates by their weighted score in plain code (no AI in the sorting). Then run ONE comparative model call over the top ~15 full CVs to fix the exact order of the best ones. Output a final ranked shortlist.
```

Step 14. Build the recruiter results dashboard

PASTE THIS TO CLAUDE CODE

```
Build the recruiter dashboard in React: show the ranked shortlist; for each candidate show the overall score, the per-criterion breakdown, the evidence quotes, any flags, and a link to open the original CV. Make it clear this is a shortlist to help the recruiter decide — not an automatic decision.
```

CHECK BEFORE MOVING ON

Run a real search end-to-end and read the shortlist in the dashboard. It should make sense to you.

Step 15. Build the golden test set and run the big test

IN PLAIN TERMS

This is the single most important task in the whole build. You assemble 300 CVs for one realistic role, decide by hand (or with an expert) who the real top 10 are, then check the engine finds them. Start collecting these CVs now — it takes longer than you think.

PASTE THIS TO CLAUDE CODE

```
Help me build a 'golden test set': a folder of 300 CVs for one realistic role, with a known expert top-10 answer key. Then create a repeatable test that runs the full pipeline on these 300 and reports: how many of the engine's top 10 match the answer key, whether any evidence came from the wrong CV, and whether every shortlisted score has real evidence. Let me re-run it with one command.
```

CHECK BEFORE MOVING ON

THE GATE: the engine's top 10 matches the answer key at the level you agreed (e.g. 8 of 10 or better), with zero cross-candidate mix-ups, and it gives the same result on three re-runs. If it fails, tell Claude Code to tune the rubric and scoring prompt — not the architecture — and re-test.

Phase 5 — Compliance & security hardening

Make it safe for a stranger's key and real candidate data. ~1 week.

Step 16. Audit log and human-in-the-loop

PASTE THIS TO CLAUDE CODE

```
Add the audit log from Phase 5: an unchangeable record of every search (the prompt, JD, weights, model used, and results). In the UI, clearly label results as decision-support that a human must act on — the tool never auto-rejects anyone.
```

Step 17. Lock it down

PASTE THIS TO CLAUDE CODE

```
Do a security pass: confirm API keys never appear in any log or error; enforce that no org can read another org's data; add rate limits and per-org quotas; add controls to delete candidate data on request and to mask names/contact details before scoring for fairness.
```

CHECK BEFORE MOVING ON

THE GATE: try to read another org's data, try to find a stored key in logs and errors — both must fail to leak. Deleting candidate data actually removes it.

Phase 6 — Pilot with design partners

Real recruiters, real roles, weekly fixes. ~2-4 weeks (light build).

IN PLAIN TERMS

You stop building features and start watching real people use it. This is where the rubric and prompts get genuinely good.

Step 18. Onboard 3-5 recruiters and watch them

- Start with independent recruiters or small agencies — they say yes fastest.
- Walk each through: add key, create job, share link or bulk-upload, run a search.
- Watch a real search over their shoulder (or a call). Write down every moment of confusion.
- Fix the top issues weekly. Tune the rubric against their real roles.
- Ask the only question that matters: did it save you time, and did you trust the shortlist?

CHECK BEFORE MOVING ON

THE GATE: at least three partners run real searches and say they'd keep using it or pay for it.

Phase 7 — Production launch & scale-readiness

Flip from pilot to a live product that won't fall over. ~1 week + ongoing.

Step 19. Make it reliable and observable

PASTE THIS TO CLAUDE CODE

```
Get us production-ready from Phase 7: add error tracking and uptime/latency monitoring (with keys redacted from all logs), turn on database backups and queue retry handling, and make the intake and processing services scale independently. Load-test a single 1,000-CV search and fix any bottlenecks.
```

Step 20. Prepare for the constant model churn

PASTE THIS TO CLAUDE CODE

```
Document and implement a simple way to add or swap an AI model purely through config (no code changes), so when the next Claude/GPT/Gemini ships I can switch in minutes. Write a one-page customer onboarding guide and an internal runbook.
```

CHECK BEFORE MOVING ON

A 1,000-CV search completes reliably; you can swap models from config; backups and monitoring are on.

Appendix A — Your Claude Code prompt habits

IN PLAIN TERMS

Reuse these phrasings constantly. They are what keep a long build clean.

- **Start of every session:** “Re-read CLAUDE.md, BUILD_PLAN.md and PROGRESS.md so you have full context.”
- **Before building:** “Before you write code, tell me your plan in 3-5 plain-English steps and wait for my OK.”
- **To understand:** “Explain what you just built in simple terms and exactly how I test it myself.”
- **When something breaks:** paste the exact error or what you saw, then “diagnose and fix this, then tell me what was wrong.”
- **To stay safe:** “never hard-code keys or model names; keep them in env/config.”
- **To save progress:** “commit this with a clear message and update PROGRESS.md.”
- **Build thin first:** “build the simplest end-to-end version that works for ONE job and ONE CV before adding more.”

Appendix B — The order matters

Foundations → Intake → Parsing → **Scoring** → **Rank/Refine (the 300→top-10 gate)** → Hardening → Pilot → Launch. Everything before the scoring engine is plumbing; everything after is trust and scale. If you only get one thing perfect, make it Phases 3 and 4.

Appendix C — Three things people skip and regret

- **The golden test set.** Without it you're guessing whether it works. Build it (Step 15) and re-run it after every change.
- **The evidence-quote check.** Spot-checking that quotes really exist in the CV is your best lie-detector for the AI.
- **The key vault done properly.** You're holding strangers' API keys. Encrypt, never log, never display. Get this right in Phase 0, not later.

Build the thin slice. Test it against the golden set. Then widen. That rhythm carries you all the way to launch.